

Lab Assignment-9.1

Problem 1: Consider the following Python function:

```
def find_max(numbers):
    return max(numbers)
```

Task:

- Write documentation for the function in all three formats:
 - (a) Docstring
 - (b) Inline comments
 - (c) Google-styled documentation
- Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
- Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

SCREENSHOT OF GENERATED CODE:

```

1  # Docstring Documentation with complete code
2  def find_max(numbers):
3      """
4      This function takes a list of numbers as input and returns the maximum value from the list.
5
6      Parameters:
7      numbers (list): A list of numerical values.
8
9      Returns:
10     The maximum value from the input list.
11
12     """
13     # Check if the input list is empty
14     if not numbers:
15         raise ValueError("The input list cannot be empty.")
16
17     # Use the built-in max function to find the maximum value in the list
18     return max(numbers)
19
20 # Example usage
21 numbers = [3, 1, 4, 1, 5, 9]
22 max_value = find_max(numbers)
23 print(f"The maximum value in the list is: {max_value}")
24
25 # Inline Documentation with complete code
26 def find_max(numbers):
27     """
28     This function takes a list of numbers as input and returns the maximum value from the list.
29
30     Parameters:
31     numbers (list): A list of numerical values.
32
33     Returns:
34     The maximum value from the input list.
35
36     """
37     # Check if the input list is empty
38     if not numbers:
39         raise ValueError("The input list cannot be empty.")
40
41     # Use the built-in max function to find the maximum value in the list
42     return max(numbers)
43
44 # Example usage
45 numbers = [3, 1, 4, 1, 5, 9]
46 max_value = find_max(numbers)
47 print(f"The maximum value in the list is: {max_value}")
48
49 # Google-Style Documentation with complete code
50 def find_max(numbers):
51     """
52     Finds the maximum value in a list of numbers.
53
54     Args:
55         numbers (list): A list of numerical values. The list should not be empty.
56
57     Returns:
58         The maximum value from the input list.
59
60     Raises:
61         ValueError: If the input list is empty.
62
63     """
64     if not numbers:
65         raise ValueError("The input list cannot be empty.")
66
67     return max(numbers)
68
69 # Example usage
70 numbers = [3, 1, 4, 1, 5, 9]
71 max_value = find_max(numbers)
72 print(f"The maximum value in the list is: {max_value}")
    
```

EXPLANATION:

The function `find\_max` takes a list of numbers as input and returns the maximum value from that list. It first checks if the input list is empty and raises a `ValueError` if it is. If the list is not empty, it uses the built-in `max` function to find and return the maximum value. The function is documented using three different styles: docstring documentation, inline documentation, and Google-style documentation, providing clear explanations of the function's purpose, parameters, return value, and potential exceptions.

Style	Advantages	Disadvantages	Suitable Use Cases
Docstring (standard)	Simple, readable, built into Python help tools	Less structured than formal styles	Small scripts, educational code
Inline comments	Explain logic step-by-step	Can clutter code, not accessible via <code>help()</code>	Complex algorithms needing explanation
Google-style	Highly structured, professional, IDE-friendly, good for teams	Slightly longer to write	Libraries, APIs, production code

Problem 2: Consider the following Python function:

```
def login(user, password, credentials):
```

```
    return credentials.get(user) == password
```

Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

SCREENSHOT OF GENERATED CODE:

```
def login(user, password, credentials):
    """
    Checks if the provided username and password match the credentials stored in a dictionary.

    Parameters:
        user (str): The username to be checked.
        password (str): The password to be checked.
        credentials (dict): A dictionary containing username-password pairs.

    Returns:
        bool: True if the username and password match the credentials, False otherwise.
    """
    return credentials.get(user) == password

# Example usage
credentials = {'user1': 'pass1', 'user2': 'pass2'}
print(login('user1', 'pass1', credentials)) # Output: True
print(login('user1', 'wrongpass', credentials)) # Output: False

def login(user, password, credentials):
    """
    Checks if the provided username and password match the credentials stored in a dictionary.

    Parameters:
        user (str): The username to be checked.
        password (str): The password to be checked.
        credentials (dict): A dictionary containing username-password pairs.

    Returns:
        bool: True if the username and password match the credentials, False otherwise.
    """
    return credentials.get(user) == password

# Example usage
credentials = {'user1': 'pass1', 'user2': 'pass2'}
print(login('user1', 'pass1', credentials)) # Output: True
print(login('user1', 'wrongpass', credentials)) # Output: False

def login(user, password, credentials):
    """
    Checks if the provided username and password match the credentials stored in a dictionary.

    Parameters:
        user (str): The username to be checked.
        password (str): The password to be checked.
        credentials (dict): A dictionary containing username-password pairs.

    Returns:
        bool: True if the username and password match the credentials, False otherwise.
    """
    return credentials.get(user) == password

# Example usage
credentials = {'user1': 'pass1', 'user2': 'pass2'}
print(login('user1', 'pass1', credentials)) # Output: True
print(login('user1', 'wrongpass', credentials)) # Output: False
```

EXPLANATION:

The function `login` takes three parameters: `user`, `password`, and `credentials`. It checks if the provided username and password match the credentials stored in a dictionary. The function uses the `get` method of the dictionary to retrieve the password associated with the given username and compares it with the provided password. If they match, it returns `True`; otherwise, it returns `False`. The function is documented using three different styles: docstring, inline comments, and Google-style.

documentation, inlined documentation, and Google-style documentation, providing clear explanations of the function's purpose, parameters, return value, and potential exceptions.

Style	Advantages	Disadvantages	Best Use Cases
Docstring (standard)	Simple, readable, built into Python help tools	Less structured for large projects	Small scripts, student projects
Inline comments	Explain logic directly near code	Not visible via <code>help()</code> , can clutter code	Complex internal logic explanation
Google-style	Structured, professional, IDE-friendly, great for teams	Slightly longer to write	Libraries, APIs, collaborative projects

Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named `calculator.py` and demonstrate automatic documentation generation. Instructions:

- Create a Python module `calculator.py` that includes the following functions, each written with appropriate docstrings:
 - `oadd(a, b)` – returns the sum of two numbers
 - `osubtract(a, b)` – returns the difference of two numbers
 - `omultiply(a, b)` – returns the product of two numbers
 - `odivide(a, b)` – returns the quotient of two numbers
- Display the module documentation in the terminal using Python's documentation tools.
- Generate and export the module documentation in HTML format using the `pydoc` utility, and open the generated HTML file in a web browser to verify the output.

SCREENSHOT OF GENERATED CODE:

The screenshot shows a code editor with the following content:

```
1 #Lab 3-1
2 #Problem-3
3
4 """
5 calculator.py - Simple calculator module with basic operations.
6 """
7
8 def add(a, b):
9     """Return the sum of a and b."""
10    return a + b
11
12 def subtract(a, b):
13     """Return the difference of a and b."""
14    return a - b
15
16 def multiply(a, b):
17     """Return the product of a and b."""
18    return a * b
19
20 def divide(a, b):
21     """Return the quotient of a and b.
22
23     Raises:
24         ValueError: If b is zero.
25     """
26     if b == 0:
27         raise ValueError("Cannot divide by zero")
28     return a / b
29
30
```

The terminal output shows the help for the calculator module:

```
Help on module calculator:
NAME
  calculator - calculator.py - Simple calculator module with basic operations.
FUNCTIONS
  add(a, b)
    Return the sum of a and b.
  divide(a, b)
    Return the quotient of a and b.
    Raises:
      ValueError: If b is zero.
  multiply(a, b)
    Return the product of a and b.
  subtract(a, b)
    Return the difference of a and b.
FILE
  /Users/navedahmedshah/Documents/3-2/AIAC/calculator.py
Help on calculator line 1/24 (END) (press h for help or q to quit)
```

[index](#)

calculator </Users/navedahmedshaik/Documents/3-2/AIAC/calculator.py>

calculator.py - Simple calculator module with basic operations.

Functions

add(a, b)

Return the sum of a and b.

divide(a, b)

Return the quotient of a and b.

Raises:

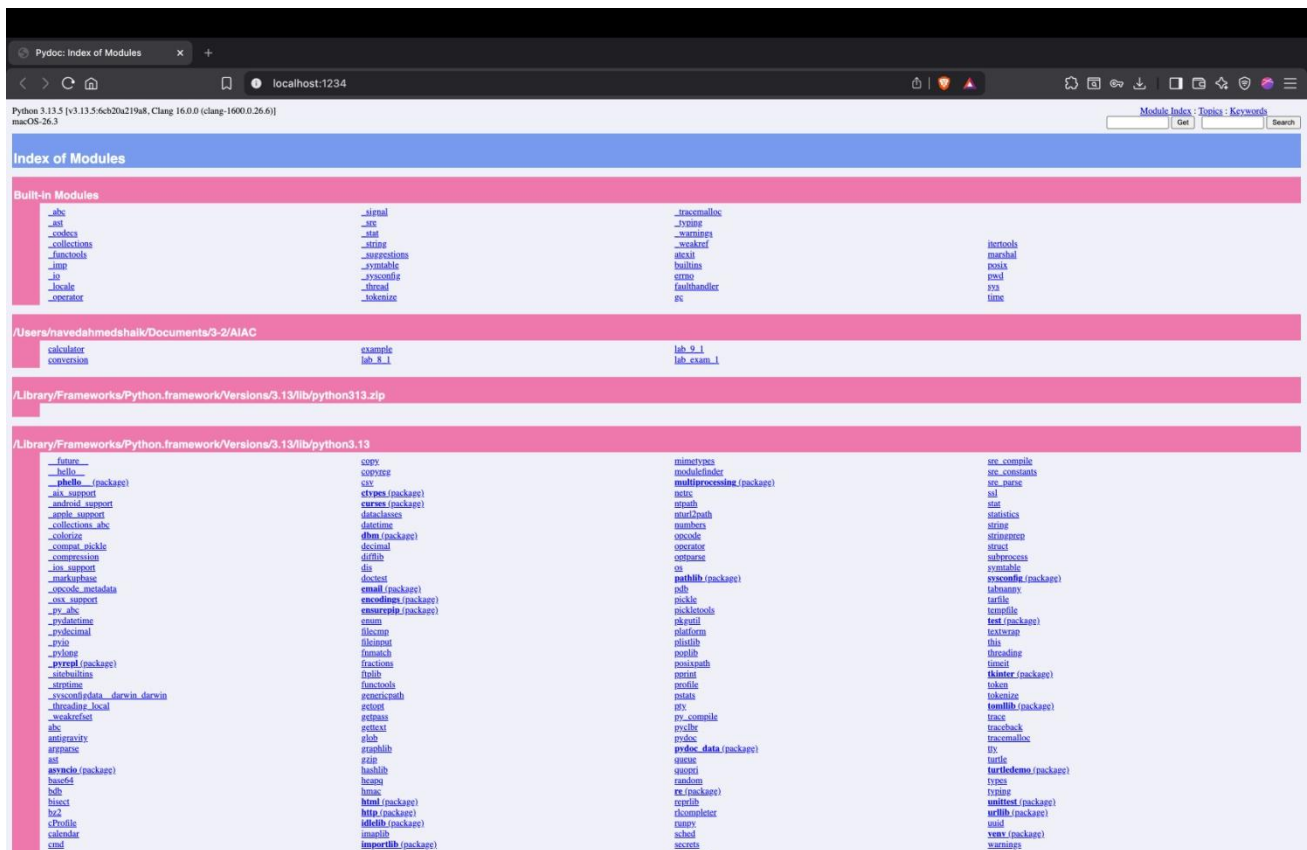
ValueError: If b is zero.

multiply(a, b)

Return the product of a and b.

subtract(a, b)

Return the difference of a and b.



Problem4:ConversionUtilitiesModule

Task:

1. Write a module named conversion.py with functions:
 - o decimal_to_binary(n)
 - o binary_to_decimal(b)
 - o decimal_to_hexadecimal(n)
2. Use Copilot for auto-generating docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

SCREENSHOTOFGENERATEDCODE:

The screenshot shows a VS Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'AIAC' with several files, including 'conversion.py'. The 'conversion.py' file is open in the editor, showing the following code:

```

1 #lab 8.1.py
2 #Problem 4
3 """
4 conversion.py
5
6 Utility functions for number system conversions:
7 - Decimal to Binary
8 - Binary to Decimal
9 - Decimal to Hexadecimal
10 """
11
12 def decimal_to_binary(n):
13     """Convert a decimal integer to its binary string representation."""
14     if n < 0:
15         raise ValueError("Only non-negative integers allowed")
16     return bin(n)[2:]
17
18
19 def binary_to_decimal(b):
20     """Convert a binary string to its decimal integer representation."""
21     if not all(ch in "01" for ch in b):
22         raise ValueError("Input must be a binary string")
23     return int(b, 2)
24
25
26 def decimal_to_hexadecimal(n):
27     """Convert a decimal integer to its hexadecimal string representation."""
28     if n < 0:
29         raise ValueError("Only non-negative integers allowed")
30     return hex(n)[2:].upper()
31

```

The terminal at the bottom shows the output of running the script. It displays help information for the 'conversion' module, including a description of the utility functions and a list of functions: 'decimal_to_binary', 'binary_to_decimal', and 'decimal_to_hexadecimal'. The terminal also shows the file path: '/Users/navedahmedshah/Documents/3-2/AIAC/conversion.py'.

[illegible]

[index](#)

conversion </Users/navedahmedshaik/Documents/3-2/AIAC/conversion.py>

conversion.py

Utility functions for number system conversions:

- Decimal to Binary
- Binary to Decimal
- Decimal to Hexadecimal

Functions

binary_to_decimal(b)

Convert a binary string to its decimal integer representation.

decimal_to_binary(n)

Convert a decimal integer to its binary string representation.

decimal_to_hexadecimal(n)

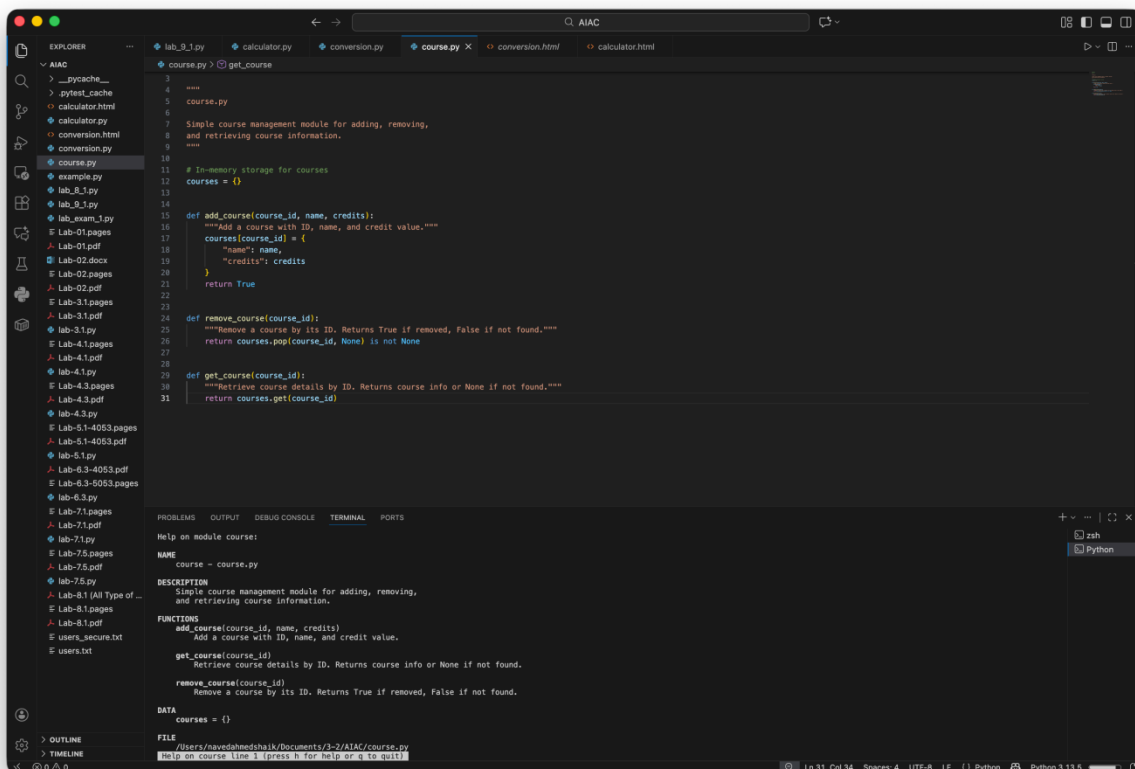
Convert a decimal integer to its hexadecimal string representation.

Problem5–CourseManagement Module

Task:

1. Create a module `course.py` with functions:
 - o `add_course(course_id, name, credits)`
 - o `remove_course(course_id)`
 - o `get_course(course_id)`
2. Add docstrings with Copilot.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

SCREENSHOT OF GENERATED CODE:



[index](#)
course </Users/navedahmedshaik/Documents/3-2/AIAC/course.py>.

course.py

Simple course management module for adding, removing and retrieving course information.

Functions

add_course(course_id, name, credits)
Add a course with ID, name, and credit value.

get_course(course_id)
Retrieve course details by ID. Returns course info or None if not found.

remove_course(course_id)
Remove a course by its ID. Returns True if removed, False if not found.

Data

courses = { }